Tutorial Calculadora - Atualizado

Este documento serve como um guia completo de requisitos, regras de negócio e competências técnicas necessárias para reconstruir uma calculadora do zero, incluindo recursos modernos de funções unárias e persistência de dados.

1. Passo a Passo de Como Montar a Calculadora

1.1 Botões que precisam ter nesse projeto:

- **Botões Padrões:** 0 a 9, "." (ponto decimal), "=" (igual), "C" (Clear - limpa o último caractere ou digitação) e "CE" (Clear Entry - limpa a entrada atual).
- **Operadores:** Multiplicação (*), Divisão (/), Soma (+) e Subtração (-).
- **Funções Avançadas:** Porcentagem (%), potência de grau 2 (x^2), raiz quadrada (√) e painel/botão de histórico.

1.2 Regras de Negócio e Validações

1.2.1 Comportamento Operacional Padrão

- **Substituição de operadores:** Se um operador matemático (+, -, *, /) for digitado imediatamente após outro, o último operador introduzido deve substituir o anterior (exemplo: a sequência 5 + * deve ser corrigida dinamicamente para 5 *).
- **Prevenção de múltiplos pontos:** Cada número em edição só pode conter um único ponto decimal. A introdução de sequências inválidas (como 5.3.2) deve ser estritamente bloqueada (utilizando Expressões Regulares - RegEx).
- **Substituição após cálculo (Controlo de Estados):** Se o utilizador pressionar a tecla = e, em seguida, digitar um dígito (0-9), o visor deve ser totalmente limpo e um novo cálculo iniciado. Caso digite um operador matemático após o =, a calculadora deve manter o resultado anterior no visor e utilizá-lo como base para a nova operação.

1.2.2 Tratamento de Erros e Exceções

Cenário / Operação Inválida	**Mensagem de Erro no Visor**	**Ação de Recuperação (Próximo Clique)**
Divisão por zero (ex: 5 / 0 ou através do modificador 1/x)	Erro: divisão por zero	Qualquer botão pressionado limpa o visor e reinicia o estado.
Raiz quadrada de número	Erro: Entrada inválida	Qualquer botão pressionado

Cenário / Operação Inválida	Mensagem de Erro no Visor	Ação de Recuperação (Próximo Clique)
negativo (ex: μ(-9))		limpa o visor e reinicia o estado.
Erro genérico de sintaxe ou processamento interno	Erro	Qualquer botão pressionado limpa o visor e reinicia o estado.

1.2.3 Funções Avançadas e Modificadores Instantâneos

- **Potência Quadrada (x²):** Eleva o número em exibição diretamente ao quadrado (ex: 5 transforma-se instantaneamente em 25).
- **Raiz Quadrada (√x):** Calcula a raiz quadrada do valor atual, respeitando a validação de erro para números negativos descrita na tabela acima.
- **Fração / Inverso (1/x):** Divide o número 1 pelo valor exibido no visor (ex: 4 passa a ser 0.25). Dispara erro se o valor for zero.
- **Lógica de Porcentagem (%):** Modifica o comportamento com base no contexto do operador principal:
    - Em multiplicações/divisões: Atua como divisor por 100. *Fórmula: A × B% = A × (B / 100)*.
    - Em somas/subtrações: Calcula o valor percentual proporcional ao primeiro operando. *Fórmula: A + B% = A + (A × (B / 100))*.

1.2.4 Recursos de Interface, Estado e Histórico

- **Registo de Histórico:** Sempre que uma equação for resolvida com sucesso através do botão =, a expressão completa (operandos + operador) acompanhada do resultado final deve ser guardada numa estrutura de dados (Array de Objetos).
- **Persistência Local (Web Storage API):** A lista de histórico deve ser serializada em JSON e armazenada no localStorage do navegador. Isto garante que o utilizador não perca os dados ao recarregar a página ou ao fechar a aplicação.
- **Reutilização de Dados:** A interface deve permitir que o utilizador clique num cálculo antigo listado no painel de histórico. Ao clicar, o resultado dessa operação deve ser injetado de volta como o operando ativo no visor principal.

2. Como Codar no JS

2.1 Conhecimentos de JavaScript (Lógica e Dados)

Para criar do zero sem depender de "receitas prontas", você precisa dominar o que já está usando e aprender algumas coisas novas:

- **Manipulação de Arrays (Listas):** Para fazer um histórico, você precisará salvar cada operação realizada. Conhecer métodos como .push() (para adicionar ao histórico) e .map() ou .forEach() (para renderizar esse histórico na tela) é essencial.
- **Armazenamento Local (Web Storage API):** Se você quer que o histórico não suma quando a página for atualizada, precisará aprender a usar o localStorage.setItem() e localStorage.getItem().
- **Expressões Regulares (RegEx):** Seu código atual usa /^[0-9]$/.test(). Entender como criar esses padrões ajuda muito a validar o que o usuário digita.
- **Substitutos para o eval():** O eval() funciona, mas é uma má prática no mundo real por motivos de segurança e performance. Aprender a criar um interpretador simples (usando loops ou funções matemáticas nativas) ou usar a biblioteca math.js vai elevar o nível do seu código.
- **Manipulação de Strings:** Métodos como .slice(), .split() e .replace() são usados o tempo todo para limpar e formatar o que aparece na tela da calculadora.

2.2 Conhecimentos de DOM (A Interface com o Usuário)

O DOM (Document Object Model) é como o JavaScript conversa com o HTML.

- **Event Delegation (Delegação de Eventos):** Você já usa isso muito bem ao colocar o addEventListener no teclado em vez de colocar em cada botão individualmente. Continue dominando o uso do evento.target.
- **Criação Dinâmica de Elementos:** Para exibir o histórico na tela, você precisará criar novos elementos HTML (como ou <li>) dinamicamente usando document.createElement() e appendChild().
- **Gerenciamento de Estados:** Entender que a calculadora tem um "estado" (ex: o número atual, o número anterior, o operador selecionado e o histórico) e que a tela apenas reflete esse estado.

2.3 Conhecimentos de HTML e CSS (Para mudar o Template)

Como você mencionou que vai mudar o template, o foco aqui é a estrutura e o design responsivo:

- **CSS Grid Layout:** É a melhor ferramenta do CSS para criar teclados de calculadora. Com poucas linhas de código, você alinha linhas e colunas perfeitamente.
- **Flexbox:** Excelente para posicionar o visor (display) e a área do histórico ao lado ou abaixo da calculadora.
- **Variáveis CSS (Custom Properties):** Facilitam muito a criação de temas (como um modo escuro/claro para a sua calculadora).
- **Semântica HTML:** Usar las tags corretas (<main>, <section>, <output> para o visor) melhora a acessibilidade do seu projeto.

Estrutura do Histórico (Exemplo Prático)

Para te dar um norte de como aplicar isso, a lógica do histórico funciona basicamente assim:

1. Criar uma lista vazia: let historico = [];

2. Antes de mostrar o resultado no =, salvar a expressão e o resultado.
3. Salvar no navegador: localStorage.setItem('historicoCalculadora', JSON.stringify(historico));
4. Criar uma função que lê essa lista e gera o HTML na tela para o usuário ver.